



JAIN
DEEMED-TO-BE UNIVERSITY

SCHOOL OF
COMPUTER
SCIENCE AND IT

DEPARTMENT OF CS & IT
Programme: MCA

Activity-1
CLOUD ANALYTICS
25MCASC302

Name: SHIVAM AKASH

USN: 25MCAR0216

Semester: 3Rd Semester

Branch: MCA-SCT

Submitted to: Dr. Prabhakaran Abraham
Associate Professor
School of CS & IT

1. Introduction and Objectives

The Azure Portfolio is a personal and professional portfolio website that I designed and developed using HTML, CSS, and JavaScript, and subsequently deployed publicly through Microsoft Azure App Service. The project was undertaken as part of Cloud Computing Activity 1, with the primary aim of gaining practical, hands-on experience in the complete lifecycle of building a web application, managing it through a version-control system, and deploying it to a real cloud environment rather than simply studying the concept in theory. GitHub is used throughout the project for source-code management, while GitHub Actions automates the deployment pipeline so that changes pushed to the repository are reflected on the live website with minimal manual intervention.

This report documents, in detail, the reasoning behind the design and development choices made for the portfolio, the technologies and platforms used, the features that were implemented, the workflow followed to move the project from a local development environment to a publicly accessible cloud application, and a set of planned enhancements that would extend the portfolio into a more data-driven cloud application in the future.

The specific objectives set out for this activity were as follows:

- Develop a clean, responsive, and professionally presented personal portfolio using appropriate front-end web technologies.
- Maintain the complete project source code in a properly organized and publicly accessible GitHub repository.
- Deploy the finished portfolio to Microsoft Azure so that it can be viewed by anyone with an internet connection, without requiring any special access.
- Demonstrate a complete and repeatable cloud deployment workflow that moves from local development, through Git and GitHub, and finally into Azure App Service.
- Lay the groundwork for future enhancements, particularly around cloud-based analytics and visitor-interaction tracking.

Beyond fulfilling the requirements of the activity, the portfolio itself serves a practical purpose: it functions as a central, always-available place where my academic background, technical skill set, project work, and resume can be reviewed by recruiters, mentors, or collaborators, and it demonstrates that I can independently take a web project from an idea through to a live, cloud-hosted deployment.

Portfolio Content Overview

The portfolio is organized into a small number of clearly separated sections so that a visitor can understand who I am and what I have built within a very short amount of time. The About Me section introduces me and my professional profile in a few concise sentences. The Education section presents my academic background and qualifications in chronological order. The Skills section highlights the technical and professional skills that are most relevant to the kind of work I want to be considered for. The Projects section showcases a curated selection of the work I have completed, including this Azure Portfolio project itself. The Resume section provides direct access to a downloadable copy of my resume, and the Contact section provides the information needed to reach me, whether by email or through other professional profiles.

The portfolio has been deployed successfully as a live Azure web application, and the resulting public URL is included at the top of this report and again in the submission links section at the end. This URL can be opened directly in any modern web browser and does not require any login or special permission to view.

2. Design and Development Approach

Design Approach

The portfolio was designed as a professional web experience with a clear and predictable information hierarchy, so that a reviewer does not need to search for the information they are looking for. Personal information, education, skills, projects, resume access, and contact details are arranged in a logical top-to-bottom order that mirrors the way a recruiter or visitor is most likely to want to read through a portfolio: first understanding who the person is, then what they know, then what they have built, and finally how to get in touch. Visual consistency was treated as a priority throughout the design process, so the same colour palette, spacing rules, and typography are applied across every section of the site rather than changing from one part of the page to the next.

Particular attention was paid to making sure that the website remains readable and usable on a wide range of screen sizes, from small mobile phone screens through to large desktop monitors, since a portfolio that only works well on one type of device would limit who can actually view it comfortably. Navigation was kept simple and intuitive, with clear section headings and, where appropriate, a fixed or easily accessible menu so that a visitor can jump directly to the section that interests them the most.

Development Approach

The development of the portfolio followed a structured, step-by-step process rather than being built all at once. The overall approach can be summarised as follows:

- Create the portfolio structure using semantic HTML5, so that each section of the page is wrapped in an appropriately named element and the document remains easy to read and maintain.
- Apply CSS3 styling for layout, typography, spacing, colour, and responsiveness, ensuring that the same styling rules scale sensibly across different screen sizes.
- Use JavaScript where interactive behaviour is required, such as menu toggles, smooth scrolling between sections, or any other client-side functionality that improves the overall experience of using the site.
- Test the website locally in the browser before committing any changes, checking that all sections render correctly and that there are no obvious layout or functional issues.
- Organize the project files into a clear folder structure, and commit and push the project to the GitHub repository so that the source code is safely version-controlled.
- Deploy the project using Azure App Service, and configure a GitHub Actions workflow so that future pushes to the main branch are deployed automatically.
- Verify the live deployment by opening the Azure-generated website URL in a browser and confirming that every section of the portfolio loads and behaves as expected.

This approach meant that problems could be identified and corrected early, at the local development stage, rather than being discovered only after the site had already been deployed to Azure. It also meant that every change pushed to GitHub was, from a fairly early point in the project, automatically reflected on the live site, which made the overall development loop faster and easier to manage.

Project Structure

The repository is organized around a single entry point, `index.html`, alongside dedicated folders for stylesheets, scripts, and static assets. This separation of concerns means that the structure of the page, its visual styling, and its interactive behaviour are each kept in their own clearly named location rather than being mixed together in a single file, which makes the project considerably easier to navigate and maintain as it grows. The overall layout of the project is as follows: an `index.html` file at the root of the repository; a folder containing the CSS stylesheets that control the visual design; a folder containing the JavaScript files responsible for interactive behaviour; an `assets` folder that holds supporting files such as the downloadable resume; and a `.gitignore` file that keeps unnecessary or environment-specific files out of version control.

Implementation Principles

- A simple and maintainable front-end structure that avoids unnecessary complexity.
- Clear separation of content, styling, and scripts, so each concern can be edited independently.
- A cloud-ready deployment pipeline through GitHub and Azure that supports quick, repeatable updates.

3. Technologies, Tools and Platforms

A combination of front-end web technologies, version-control tools, and Microsoft Azure cloud services were used to design, build, and publish the portfolio. Each of these is described below, along with an explanation of the specific role it plays in the overall project.

HTML5 was used to define the underlying structure and content of the portfolio, providing the semantic skeleton on which everything else is built. CSS3 was used to control the visual design of the site, including its layout, typography, colour scheme, spacing, and responsiveness across different devices. JavaScript was used to support client-side interaction and any dynamic behaviour that the static HTML and CSS could not provide on their own, such as interactive navigation elements.

For source-code management, Git was used to provide local version control and to track every change made to the project over time, while GitHub was used to host the source code and the overall project repository in a location that is both backed up and publicly accessible. GitHub Actions was configured to automate the deployment workflow, so that once changes are pushed to the appropriate branch on GitHub, the updated site is built and published to Azure without any further manual steps being required.

On the cloud side, Microsoft Azure provides the underlying platform on which the finished website is hosted, and Azure App Service specifically hosts the live web application and exposes it through a public URL that anyone can visit.

Why Azure App Service?

Azure App Service was chosen because it provides a fully managed cloud hosting environment for the portfolio, which means that the underlying servers, patching, and infrastructure concerns are handled by Azure rather than by me directly. This allows the website to be published with a publicly accessible endpoint very quickly, and it integrates cleanly with a source-control-based deployment workflow, which was an important requirement for this activity.

Why GitHub Actions?

GitHub Actions was chosen to provide an automated continuous integration and continuous deployment, or CI/CD, workflow for the project. Once changes are pushed to the configured branch of the repository, the GitHub Actions workflow can build and deploy the website to Azure App Service automatically, which significantly reduces the amount of repeated manual deployment work that would otherwise be required every time the portfolio is updated.

Development-to-Cloud Stack

Taken together, these tools form a single, continuous pipeline that carries the project from a developer's local machine all the way to a publicly accessible website: the portfolio begins as local HTML, CSS, and JavaScript files, is tracked with Git, pushed to GitHub, picked up by a GitHub Actions workflow, deployed to Azure App Service, and finally served to the public as a live website.

4. Features and Functionalities

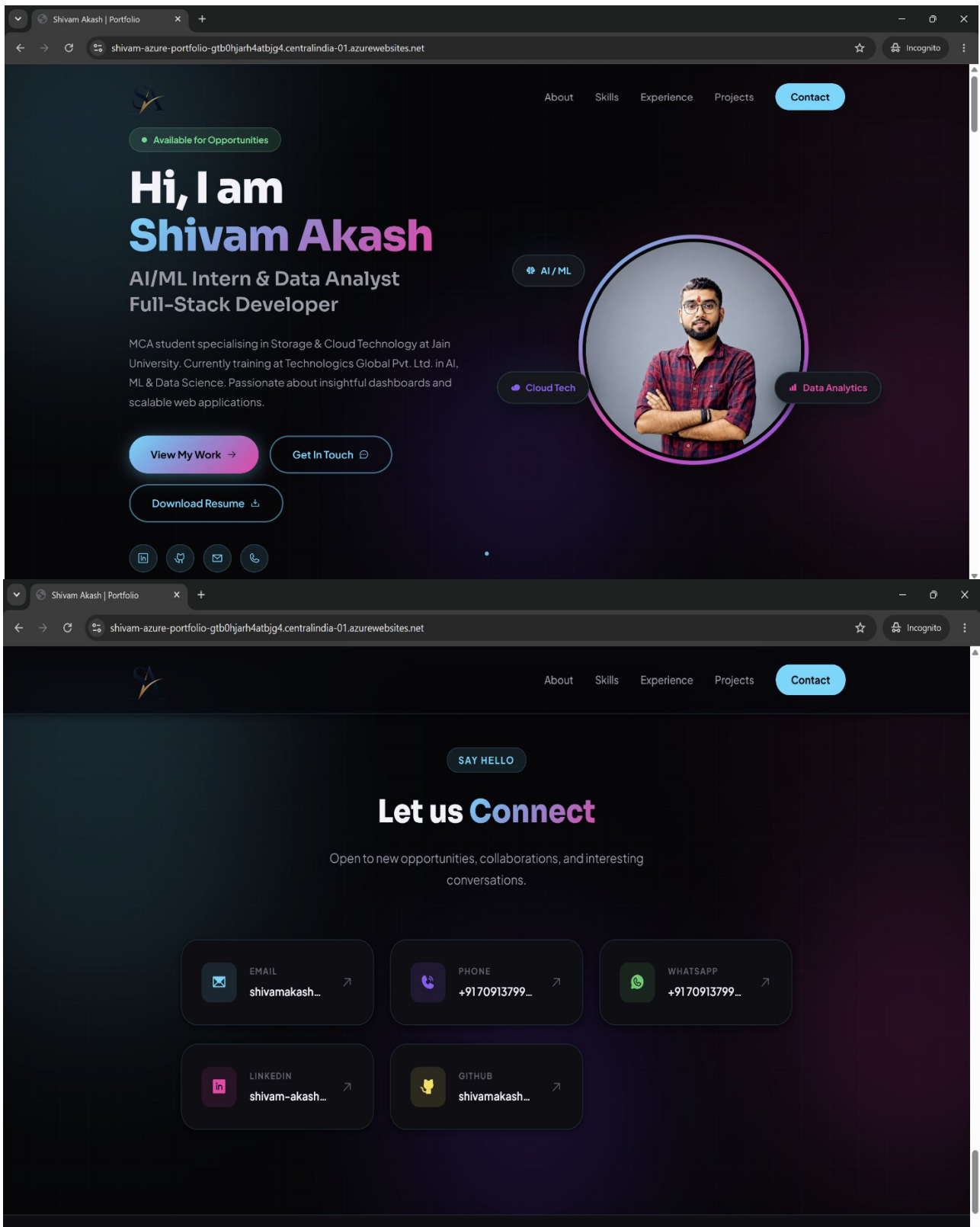
The following features have been implemented in the current version of the portfolio, covering both the content presented to visitors and the underlying infrastructure that keeps the site running and up to date:

- A professional personal and professional profile presentation, giving visitors a concise introduction to who I am.
- An education section presenting my academic background and qualifications.
- A technical skills section highlighting the tools and technologies I am comfortable working with.
- A project showcase section that presents selected work, including this Azure Portfolio project.
- Resume access directly through the portfolio, allowing visitors to download a copy of my resume.
- A contact section providing the information needed to reach me for further communication.
- Public hosting through Azure App Service, ensuring the site is available at all times to anyone with the link.
- Source-code management through GitHub, keeping the project properly version-controlled and backed up.
- An automated deployment workflow using GitHub Actions, so updates reach the live site without manual redeployment.

Together, these features satisfy the core requirement of the activity: a working, publicly accessible portfolio that is properly documented, version-controlled, and deployed through a genuine cloud workflow rather than hosted through a simple static file upload.

5. Portfolio and Deployment Screenshots

The screenshots below illustrate the completed portfolio, the organisation of the GitHub repository, and the Azure deployment configuration. Each image should be replaced with an actual screenshot taken from the live project and the corresponding GitHub and Azure dashboards before this report is finalised and submitted.



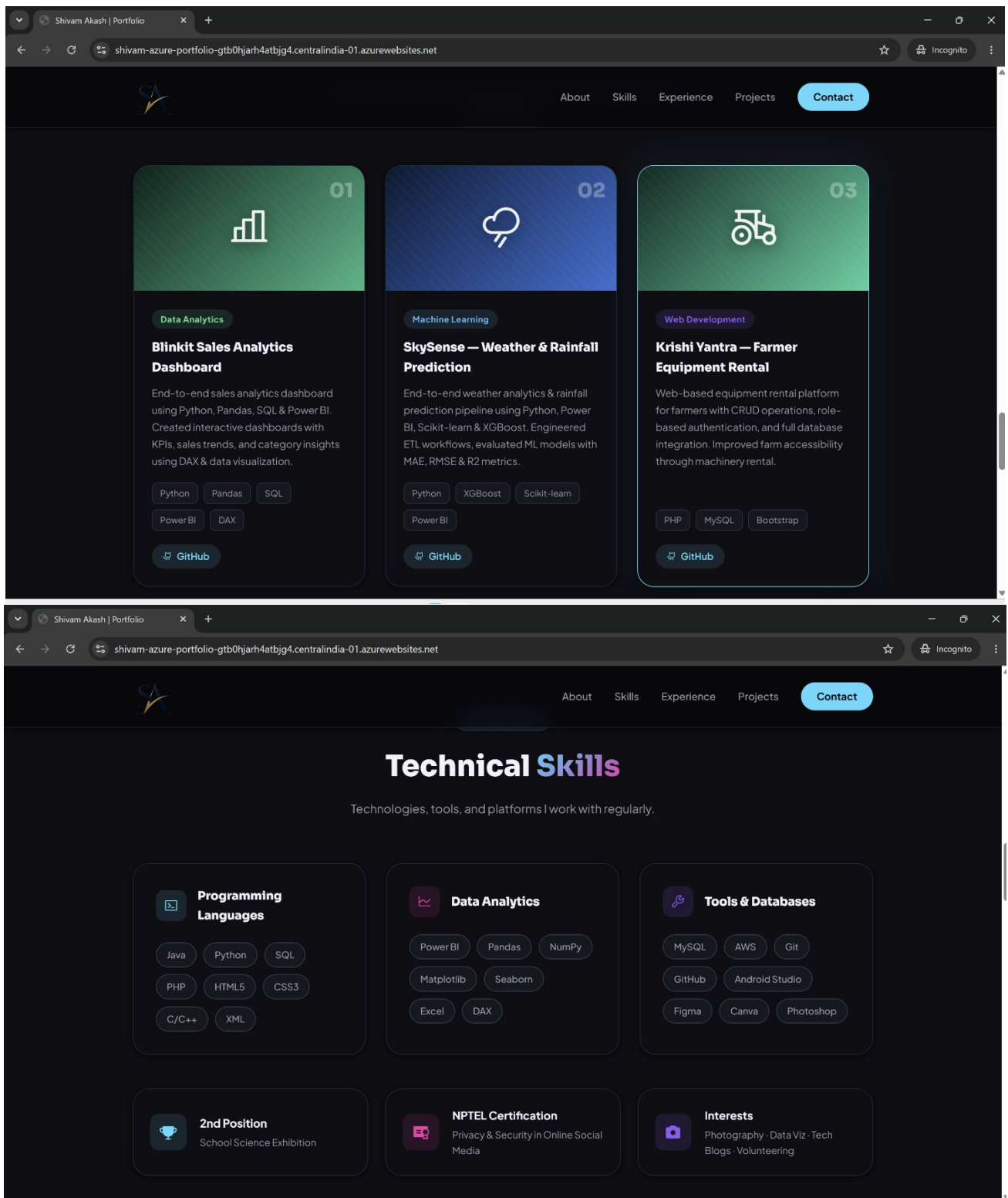


Figure 1: Homepage view of the deployed Azure Portfolio.

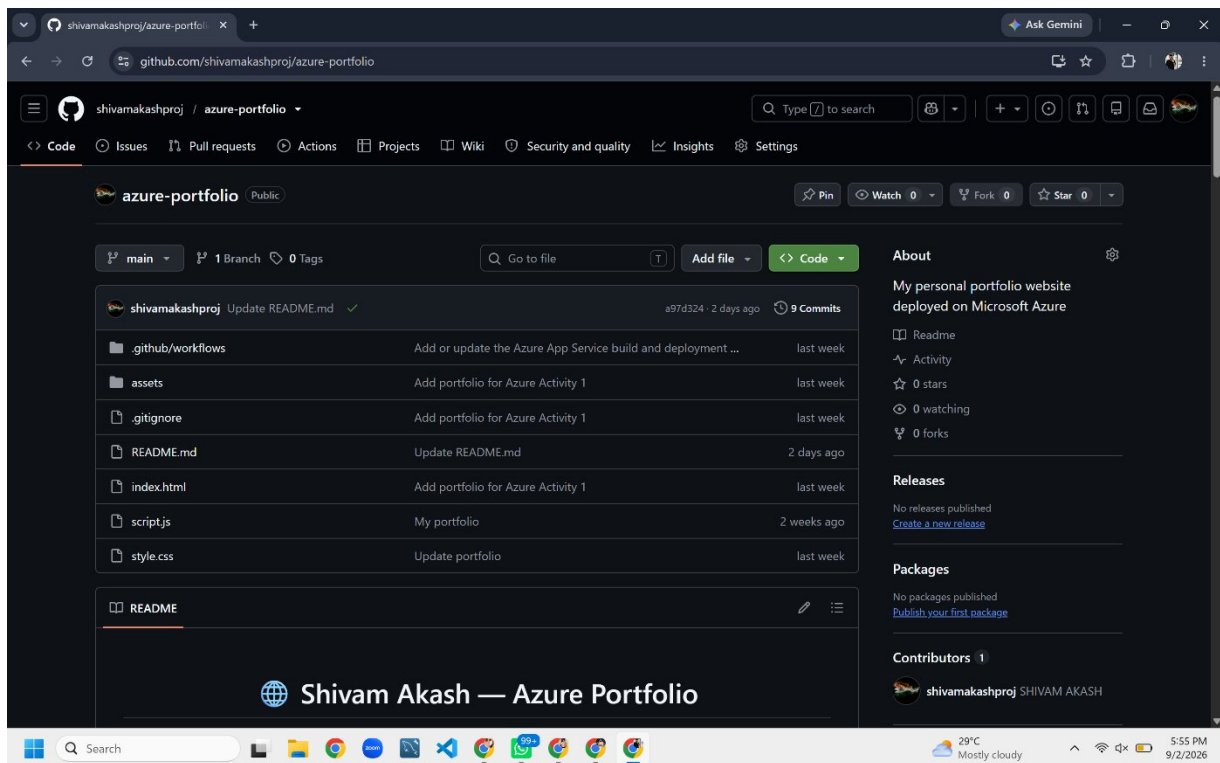


Figure 2: GitHub repository showing the organised project structure and README.

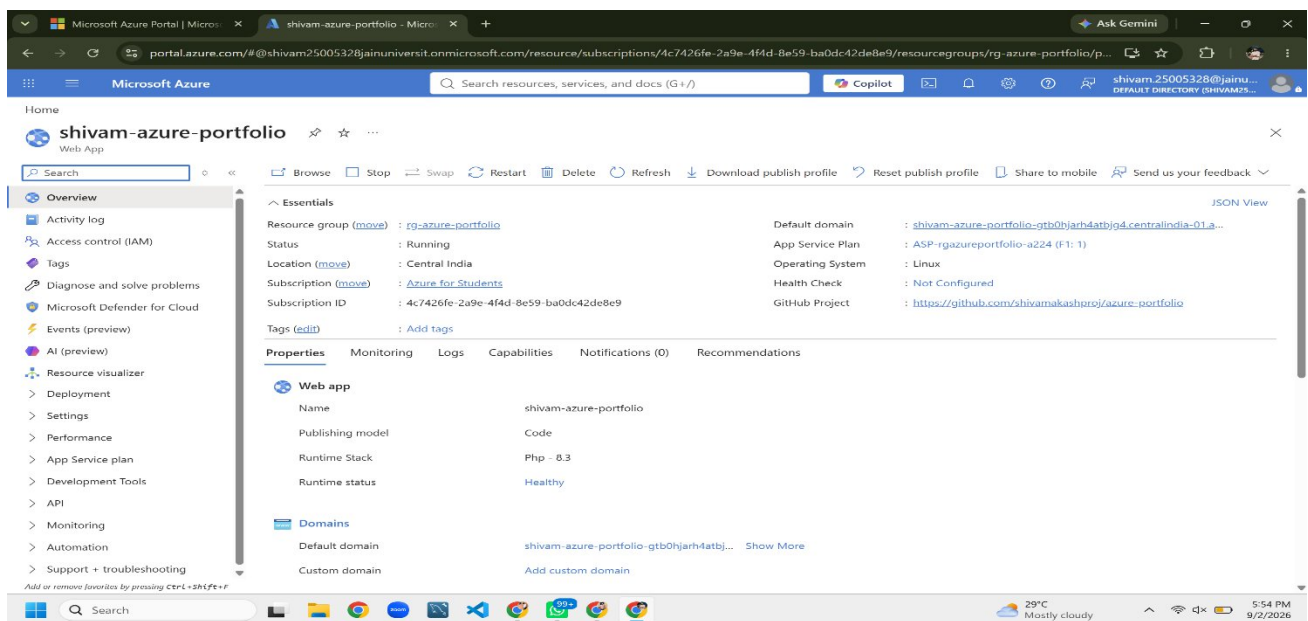


Figure 3: Azure App Service dashboard showing the running application.

6. GitHub Repository and Version Control

The complete portfolio project is maintained in GitHub under the repository named azure-portfolio, owned by the account shivamakashproj. The main branch of this repository is used both as the primary development branch and as the branch that the deployment workflow watches for changes, so any update merged into main is automatically considered for deployment to Azure App Service.

Every change made to the project is recorded using Git commits and then pushed to the remote GitHub repository. This provides a complete, readable history of how the project has evolved over time, makes it

possible to review or roll back to an earlier version if something goes wrong, and keeps the source code directly connected to the automated deployment workflow described earlier in this report. A typical update to the live site follows a small, repeatable sequence of Git commands: staging the changed files, committing them with a short descriptive message, and pushing the commit to the main branch on GitHub, after which the GitHub Actions workflow takes over and republishes the site automatically.

The repository itself is organized so that it contains the full website source code, supporting assets such as the resume file, a .gitignore file to keep unnecessary files out of version control, and project documentation. The README file included in the repository gives a concise explanation of the project, the technologies used, the Azure deployment configuration, and a link to the live deployment, so that anyone visiting the repository can quickly understand what the project is and how it was built without needing to read this full report.

7. Azure Deployment and CI/CD

The portfolio is hosted on Microsoft Azure using an Azure for Students subscription, within a dedicated resource group created specifically for this project, named rg-azure-portfolio. The web application itself runs as an Azure App Service instance named shivam-azure-portfolio, deployed in the Central India region. The application is configured to run using the PHP 8.3 runtime available through Azure App Service, and its source is connected directly to the azure-portfolio GitHub repository and its main branch, so that Azure and GitHub remain in sync with one another.

The overall deployment process followed to bring the site online can be described in a series of stages. First, the portfolio source code is maintained and updated in GitHub. Azure App Service is then configured specifically for the portfolio application, including setting the correct runtime and region. GitHub is connected to Azure through the App Service deployment configuration, and a GitHub Actions workflow file is added to the repository to define exactly how the build and deployment steps should run. Once this workflow is triggered, either automatically by a push to the main branch or manually, it builds and publishes the project to Azure. The resulting deployment status can then be checked directly within the Azure Deployment Center, and finally, the live Azure URL is tested in a browser to confirm that the deployment has completed successfully and that the site is behaving as expected.

This deployment completed successfully, and the portfolio is now available publicly through its Azure App Service URL, which is included at the top of this report and again in the submission links section below. Because the deployment pipeline is automated through GitHub Actions, any future update to the portfolio only requires a normal Git push to the main branch, after which Azure will rebuild and republish the site without any further manual steps.

8. Future Enhancement — Azure Analytics Tracker

A planned future enhancement for this portfolio is the addition of an Azure Analytics Tracker. This feature would extend the currently deployed website by collecting, storing, processing, and ultimately visualising data about portfolio visitors and how they interact with the site, using a combination of Azure services and, potentially, Power BI for the final reporting and dashboard layer.

The data that this future feature is intended to capture includes the overall visitor count and the number of visits to individual pages, contact form submissions made by visitors, clicks on individual projects within the project showcase, resume download activity, and a number of other basic engagement metrics that would help me understand how people are actually using the portfolio rather than simply assuming how it is being used.

At a high level, the planned architecture for this feature would begin at the portfolio website itself, where visitor and interaction events would be captured and sent to a dedicated analytics or backend service. That service would then store the incoming data in Azure Storage, after which it would be processed and made available to a dashboard, most likely built using Power BI, where the data could be reviewed and analysed. In addition to this reporting pipeline, the plan includes building an admin panel that would allow visitor statistics, contact submissions, and project-interaction data to be reviewed directly, without needing to open a separate analytics tool. Once implemented, this enhanced version of the portfolio, complete with analytics, would remain publicly available for demonstration and review, in the same way that the current version is.

This enhancement is not yet part of the current live deployment, but it has been planned in enough detail that it represents a realistic and achievable next step for the project, and it demonstrates an awareness of how a simple static portfolio can be extended into a more complete, data-informed cloud application over time.

9. Conclusion

The Azure Portfolio project demonstrates the complete, practical workflow involved in developing a personal web portfolio, managing its source code through GitHub, and deploying it as a genuinely public cloud application using Microsoft Azure App Service together with GitHub Actions. Rather than treating cloud deployment as an abstract concept, this activity required actually creating an Azure account, configuring a real App Service instance, connecting it to a GitHub repository, and confirming that the resulting site is reachable by anyone on the internet.

Beyond meeting the specific requirements of Cloud Computing Activity 1, the project has provided a solid, working foundation that can be built upon in future coursework or personal projects, particularly in the areas of analytics, cloud data storage, data visualisation, and basic web application administration, as outlined in the future enhancement section above. The skills developed while completing this activity, including using Git and GitHub for version control, configuring and deploying an Azure App Service, and setting up an automated GitHub Actions deployment pipeline, are directly transferable to future, larger-scale cloud projects.

10. Submission Links

GitHub Repository: <https://github.com/shivamakashproj/azure-portfolio>

Live Portfolio: <https://shivam-azure-portfolio-gtb0hjarh4atbjg4.centralindia-01.azurewebsites.net>